

untangle::executor

Generated by Doxygen 1.18.0

1 executor	1
2 Namespace Index	7
2.1 Namespace List	7
3 Class Index	9
3.1 Class List	9
4 File Index	11
4.1 File List	11
5 Namespace Documentation	13
5.1 untangle Namespace Reference	13
5.1.1 Detailed Description	13
6 Class Documentation	15
6.1 untangle::executor< actionT > Class Template Reference	15
6.1.1 Detailed Description	15
6.1.2 Constructor & Destructor Documentation	16
6.1.2.1 executor()	16
6.1.2.2 ~executor()	16
6.1.3 Member Function Documentation	17
6.1.3.1 add_action()	17
6.1.3.2 add_task()	18
6.1.3.3 start()	19
6.1.3.4 stop()	19
6.1.4 Member Data Documentation	19
6.1.4.1 on_task_error	19
7 File Documentation	21
7.1 executor.hpp	21
Index	25

Chapter 1

executor

c++ thread pool executor based on async execution

The interface is one header that exposes a pool of workers, each one an **async** execution in continuous mode, fed by a shared queue of tasks.

This is the prototype, imported. It was written in `async/prototypes/` to answer whether execution is a usable building block for a pool — it is — and it is being hardened here. The behaviour below is what the tests state today, not a settled interface.

requirements

C++23, and the `async` submodule:

```
git submodule update --init --recursive
```

`async` carries its own actuator submodule, which is why the clone has to be recursive. Both are header-only; nothing is built or installed.

The headers report warnings with `std::println`, so they need a standard library that provides `<print>` — GCC 14 or newer.

example

```
#include <executor.hpp>
#include <functional>

int main() {
    untangle::executor<std::function<void(void)>> pool(4);
    pool.start();

    for (int i = 0; i < 100; ++i) {
        pool.add_action([i] { work(i); });
    }

    return 0; // ~executor finishes what was added, then stops the workers
}
```

the action type

The pool is a template on the callable it runs, the way an execution is — and on the same parameter name, because both kinds of queued work are that one callable. One pool runs one signature, and the caller names it:

```
untangle::executor<std::function<int(void)>> pool(4); // returns are run and dropped
```

The action type does not have to be a `std::function`. What an execution requires of it is a nested `result_type`, which it reads rather than deduces, so any callable declaring one will do — and a pool built on such a callable never touches `std::function::result_type`, which the standard removed in C++20 and both `libc++` and `libstdc++` still supply by grace rather than contract.

```
struct my_task {
    using result_type = void;
    void operator()() const;
};
```

```
untangle::executor<my_task> pool(4);
```

An action type may take arguments, and `add_action()` binds them the way its namesake `execution::add_action()` does — at the door, where the caller still holds them:

```
untangle::executor<std::function<int(int)>> pool(4);
pool.start();
pool.add_action([](int n) { return n * 2; }, 21);
```

What waits for a worker is one callable carrying its own arguments, so nothing about a task changes between the queue and the worker that runs it. The arguments are copied at `add_action()` and handed to the task as the pool's own lvalues when it runs: a task is called later, possibly much later, and what the caller passed may be gone by then. A task taking `int&` therefore mutates the pool's copy, which the caller never sees.

A task that cannot be called with the arguments given is refused by a `static_assert` in `add_action()`, rather than by an error from inside the binding.

tasks: work that reports back

Everything above queues work through `add_action()`, which is fire and forget: it runs, what it returns is dropped, and the caller hears nothing more about it. `add_task()` carries the callback it must notify:

```
untangle::executor<std::function<int(int)>> pool(4);
pool.start();

// the callback comes last, after the arguments the task is bound to
pool.add_task([](int n) { return n * 2; }, 21, [](int result) { /* result == 42 */ });
```

Work that returns void still reports that it finished, with a callback taking nothing — finished is the message and the result is optional:

```
untangle::executor<std::function<void(void)>> pool(4);
pool.add_task([] { /* ... */ }, [] { /* finished */ });
```

The callback is the last argument, and the signature cannot say so. A parameter pack cannot be followed by a deducible parameter, so it arrives inside the arguments and `untangle::bind_task()` splits it off. It must be callable with the task's result and return nothing — or callable with nothing at all, when the task returns void. A missing or unusable one is a compile error rather than silence.

Both doors share one queue, and the order out is the order in. An action posted before a task runs before it. The pool can promise that because it owns one container; `async::execution`, which the workers are, fires every action in a pass before any task, so its ordering across the two kinds differs. Only the pool's own order is a promise to a caller of the pool.

`pending()` counts both kinds, as it always counted work waiting for a worker.

A refused task does not notify. `add_task()` answers false under exactly the conditions `add_action()` does — before `start()`, after `stop()`, once the destructor has begun, or if the worker it was offered to has stopped — and a refused task is destroyed rather than queued. The answer is the whole report, so a caller waiting on the callback rather than on the answer would wait for ever. The same is true of work still queued when a stopped pool is destroyed: it never runs, so it never notifies, and the count on `stderr` is all there is.

Finished does not mean failed. A task that throws is not notified: there is no result to hand over, and for a void task no completion to report either. What it threw goes to `on_task_error`, as a failing action's does. And because a callback runs inside the same try as the task that owns it, `on_task_error` can fire for a task that in fact succeeded — the work was done and only the telling failed.

A callback runs on a worker's thread, inside that worker's drain, and several may be running at once on different workers. It is the same rule `on_task_error` carries, for the same reason: nothing may escape it.

how work is placed

N executions run in continuous mode behind a shared `std::deque` of tasks.

Nothing overtakes. A task goes straight to a free worker only when the shared queue is empty; otherwise it joins the back of the queue. Finding a free worker does not let a task jump a queue that already has work in it.

Workers pull rather than being pushed to. When one reports its queue finished — `on_finished`, raised by execution the moment its list empties — it takes the front of the shared queue. That is what spreads the work, with no scheduling logic of its own: 400 tasks over 4 workers come out 100/100/100/100.

A worker is asked, not tracked. `execution::is_busy()` is what `add_action()` reads to find a free worker and what the destructor reads to decide the pool is idle. A tally kept here would be the pool's belief about its workers; this is the workers' own answer, and it cannot drift.

running, stopping, restarting

A pool is built stopped. The constructor makes the workers and wires them up; `start()` runs them. Until it is called the pool refuses work, because a task taken then would sit unrun and the destructor would wait for it. It is also the moment to assign `on_task_error`, before anything can throw.

`stop()` stops the workers, and stops the pool accepting. What a worker is already running it finishes; what is still queued stays queued. `stop()` does not wait for the workers to leave their threads — only the destructor does that, and that wait is what makes destroying the pool safe.

`start()` after `stop()` restarts it, workers and all, and the queue it was holding drains as they pick it up again. Either call is harmless twice over.

lifetime and failure

The destructor finishes what was added. It stops accepting, waits for the queue to empty and for every worker to go idle, and only then stops them. A task added before the pool goes out of scope has run by the time it does.

A pool destroyed while stopped is the exception: its queue can never empty, because only a running worker takes from it, so the destructor does not wait for it. Whatever is left is reported on stderr and dropped.

Both waits are unbounded, and both say so while they last. A task that never returns would otherwise leave the destructor silent for ever, so it reports instead — the first wait naming who is still inside a task, the second naming whose thread has not left:

```
executor: still waiting to finish after 1s - 1 queued, 2 in a task: pool_worker_0, pool_worker_1
executor: still waiting to finish after 3s - 0 queued, 1 in a task: pool_worker_1
executor: still waiting to stop after 1s - 1 not left its thread: pool_worker_1
```

The interval starts at a second and doubles to a ceiling of thirty, so a stuck teardown says something almost at once without a long one becoming a flood. The two waits are kept apart because they answer different questions: a worker can be idle and still be in its thread.

Bounding either wait was never available. Returning early while a worker is still in a task is a use-after-free, and a bounded wait only moves the hang into `~execution()`, which spins on its own running state with no timeout. Workers are detached and `async::execution` has no cancellation, so a pool cannot outlive a task it cannot interrupt. What can be fixed is the silence.

`add_action()` says whether the task was taken. It returns false before `start()`, after `stop()`, and once the destructor has begun. It returns false again if the worker it was offered to has stopped. A refused task is destroyed rather than queued, so a false answer means it will not run.

A task that is still running is refused too, and that is chosen rather than incidental. A pool shutting down is waiting for exactly that task, so it is tempting to let the work it spawns through — but then a task that re-adds itself could keep the shutdown from ever ending. One rule instead: a pool that is not running takes no work, whoever is asking. `add_action()` answers false inside the task, so it can tell.

A task that throws does not take the worker with it, and what it threw is not lost. The worker catches it and carries on with the next task; `on_task_error` is where the throw goes:

```
untangle::executor<std::function<void(void)>> pool(4);

pool.on_task_error = [](std::exception_ptr thrown) {
    try {
        std::rethrow_exception(thrown);
    } catch (const std::exception& e) {
        log(e.what());
    }
};
```

It carries a `std::exception_ptr` because a task may throw something that is not a `std::exception`, and that is the case most worth hearing about. Assign it before the first `add_action()` — a worker reads it — and expect it on a worker's thread, with more than one worker possibly inside it at once. Leave it unset and the pool warns on stderr instead, naming the worker.

That name is not unique between pools. Workers are named by index — `pool_worker_0`, `pool_worker_1` — so two pools in one process each have a `pool_worker_0`, and every warning that names one is ambiguous: the failure reports above, and the destructor's waits alike. With a single pool the name says which worker; with two it does not. Assigning `on_task_error` sidesteps it for failures, since the handler is the pool's own and the warning is replaced.

A return value is still not collected: a continuous worker keeps none. Work with something to report carries its own channel, which is what `add_task()` is for — see tasks: work that reports back above.

building the tests

```
cmake -S test -B test/build
cmake --build test/build
ctest --test-dir test/build
```

test/CMakeLists.txt fetches googletest at configure time, so the first configure needs network access.

sanitizers

EXECUTOR_SANITIZE builds the tests under a sanitizer. It is off by default, because a sanitized build is several times slower and ThreadSanitizer does not ship for every toolchain.

```
cmake -S test -B test/build-asan -DEXECUTOR_SANITIZE=address
cmake --build test/build-asan
ctest --test-dir test/build-asan
```

Accepted values are address, thread, undefined, or empty. Anything else is refused at configure time rather than passed through to the compiler.

Use a separate build directory per sanitizer. address and thread instrument the same accesses in incompatible ways and cannot be combined, so one build directory is one sanitizer.

the reference

tools/make_doc.sh builds doc/refman.pdf from the header and this README. It needs doxygen, tectonic and python3, and it fails on any doxygen warning that is not an obsolete-tag notice, so an undocumented member or a broken \ref does not reach the reference.

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	A thread pool executor built out of <code>untangle::async::execution</code> objects	13
--------------------------	---	--------------------

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

[untangle::executor< actionT >](#)
Runs actions and tasks on a fixed pool of untangle::async::execution workers [15](#)

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

executor.hpp	21
--	----

Chapter 5

Namespace Documentation

5.1 untangle Namespace Reference

A thread pool executor built out of `untangle::async::execution` objects.

Classes

- class `executor`
Runs actions and tasks on a fixed pool of `untangle::async::execution` workers.

5.1.1 Detailed Description

A thread pool executor built out of `untangle::async::execution` objects.

N workers run in continuous mode behind a shared queue. A task goes straight to a free worker only when the queue is empty; otherwise it joins the back, and a worker takes the front as it frees up.

Chapter 6

Class Documentation

6.1 `untangle::executor< actionT >` Class Template Reference

Runs actions and tasks on a fixed pool of `untangle::async::execution` workers.

```
#include <executor.hpp>
```

Public Member Functions

- `executor` (`std::size_t worker_count`)
Builds `worker_count` workers. They do not run until `start()`.
- `~executor` ()
Finishes what has been added, then stops every worker.
- `template<typename... Args>`
`bool add_action` (`actionT task`, `Args &&... args`)
Queues a task bound to `args`, or hands it to a worker that has nothing to do.
- `template<typename... Args>`
`bool add_task` (`actionT task`, `Args &&... args`)
Queues work that must report, or hands it to a worker that has nothing to do.
- `std::size_t pending` () `const`
How many tasks are waiting for a worker.
- `void start` ()
Starts every worker, and starts the pool accepting work.
- `void stop` ()
Stops every worker. What they are already running, they finish.

Public Attributes

- `std::function< void(std::exception_ptr)>` `on_task_error`
Called with whatever the work threw. Assigned by the caller.

6.1.1 Detailed Description

```
template<typename actionT>  
class untangle::executor< actionT >
```

Runs actions and tasks on a fixed pool of `untangle::async::execution` workers.

Template Parameters

actionT	The callable the pool runs. One pool runs one signature - its arguments as much as its result.
---------	--

Remarks

****Two kinds of work may be queued, and both are an actionT.**** [add_action\(\)](#) is fire and forget: what the work returns is dropped, and nothing reports that it finished. [add_task\(\)](#) carries a callback and reports what the work produced - or, for a void callable, merely that it finished. The names and the split are `async::execution's`.

6.1.2 Constructor & Destructor Documentation

6.1.2.1 executor()

```
template<typename actionT>
untangle::executor< actionT >::executor (
    std::size_t worker_count)  [inline], [explicit]
```

Builds `worker_count` workers. They do not run until [start\(\)](#).

Parameters

<code>worker_count</code>	- How many workers to run. Must not be zero.
---------------------------	--

Exceptions

<code>std::invalid_argument</code>	- <code>worker_count</code> is zero. Such a pool would take work it could never run.
------------------------------------	--

6.1.2.2 ~executor()

```
template<typename actionT>
untangle::executor< actionT >::~~executor ()  [inline]
```

Finishes what has been added, then stops every worker.

Both waits are unbounded, and both report on `stderr` while they last, naming the workers they are waiting on.

Attention

A task still running here is refused if it adds more work, even though this waits for that very task: a shutdown that took new work could be kept from ever ending by a task that re-adds itself. [add_action\(\)](#) and [add_task\(\)](#) answer false, so it can see that.

Queued work dropped here never runs, so a task dropped here never notifies, and a caller waiting on the callback rather than on the answer would wait for ever. The count on `stderr` says how much was lost, never which.

6.1.3 Member Function Documentation

6.1.3.1 add_action()

```
template<typename actionT>
template<typename... Args>
bool untangle::executor< actionT >::add_action (
    actionT task,
    Args &&... args) [inline]
```

Queues a task bound to args, or hands it to a worker that has nothing to do.

Remarks

Named for `async::execution::add_action()`, which it forwards to and behaves as: what the work returns is dropped, and nothing reports that it finished. The answer below is the last the caller hears.

The arguments are bound here, while the caller still holds them, so what waits for a worker is one callable carrying its own - a queue holds callables, and a call site cannot be queued.

Remarks

The first free worker takes it, counting from the start, so an idle pool keeps giving work to the same one: the worker that just ran a task is the warm one.

Template Parameters

Args	- The argument types to bind to task.
------	---------------------------------------

Parameters

task	- The task to run.
args	- What to bind to it. Copied here and handed over as the pool's own lvalues, so a task taking a reference is given that copy rather than the caller's object, which may be gone.

Returns

true - accepted. false - refused by a pool not started, stopped or being destroyed, or by a stopped worker; either way it will not run. A task calling this from a worker thread is answered like any other caller.

6.1.3.2 add_task()

```
template<typename actionT>
template<typename... Args>
bool untangle::executor< actionT >::add_task (
    actionT task,
    Args &&... args) [inline]
```

Queues work that must report, or hands it to a worker that has nothing to do.

The task runs exactly as one given to [add_action\(\)](#) does, and the callback it was built with is then invoked with what it returned, on the worker's thread inside the worker's drain.

Attention

The last argument is the callback, and this signature cannot say so: a pack cannot be followed by a deducible parameter, so it arrives inside args and untangle::bind_task() splits it off. It must take the task's result and return nothing - or take nothing at all for a void task, because finished is the message. A missing or unusable one is a static_assert rather than silence.

Finished does not mean failed. A task that throws is not notified - there is no result to hand over - and what it threw goes to [on_task_error](#), as a failing action's does. A caller waiting on the callback alone would wait for ever.

A refused task does not notify either, and the answer below is the whole report. An empty callback is refused too, which its type cannot say: the static_assert asks whether the callback can be called, while a default constructed std::function answers no at run time. untangle::bind_task() builds an empty task from one, and a task that can neither run nor notify is turned away here rather than queued to throw on a worker. An empty action goes the same way.

Remarks

It shares one queue with [add_action\(\)](#), and the order is the order they arrived.

Template Parameters

Args	- The argument types to bind, of which the last is the callback.
------	--

Parameters

task	- The task to run.
args	- What to bind to it, followed by the callback to notify. Copied here, as add_action() copies them, and for the same reason.

Returns

true - the task was accepted, and the callback will be notified when it has run.

false - refused, because the pool is not running, or because the task could never do its job. It will neither run nor notify.

6.1.3.3 `start()`

```
template<typename actionT>
void untangle::executor< actionT >::start () [inline]
```

Starts every worker, and starts the pool accepting work.

A pool refuses work until this is called, and refuses it again after `stop()` until this is called a second time - a stopped pool is restarted by it, workers and all. Calling it twice over is harmless.

6.1.3.4 `stop()`

```
template<typename actionT>
void untangle::executor< actionT >::stop () [inline]
```

Stops every worker. What they are already running, they finish.

Queued tasks stay unrun and a task added afterwards is refused. That holds whoever is asking, a task already running on a worker included - a pool that is not running takes no work. `start()` gives the workers a new working life. Calling it twice is harmless.

Attention

It does not wait for the workers to leave their threads. Only the destructor does that, and that wait is what makes destroying the pool safe.

6.1.4 Member Data Documentation

6.1.4.1 `on_task_error`

```
template<typename actionT>
std::function<void(std::exception_ptr)> untangle::executor< actionT >::on_task_error
```

Called with whatever the work threw. Assigned by the caller.

Nothing else reports failed work: `add_action()` and `add_task()` have both answered long before it runs. Left unset, the pool warns on `stderr` instead. It receives a `std::exception_ptr`, because the work may throw what is not a `std::exception`; rethrow it to read it.

Remarks

A task's failure arrives here as well, and so does its callback's. A task that throws is not notified, so this is the only report it makes - and because a callback runs inside the same try as its task, this can fire for a task that in fact succeeded: the work was done and only the telling failed.

Attention

It runs on a worker's thread, and several workers may be in it at once - as may several callbacks, which run on a worker's thread for the same reason. Assign it before the first `add_action()` or `add_task()`, and let nothing escape it or a callback.

The documentation for this class was generated from the following file:

- `executor.hpp`

Chapter 7

File Documentation

7.1 executor.hpp

```
00001 // Copyright (c) 2026 Nicolae Popescu. MIT License.
00002
00010 #pragma once
00011
00012 #include <algorithm>
00013 #include <async.hpp>
00014 #include <atomic>
00015 #include <chrono>
00016 #include <condition_variable>
00017 #include <cstdint>
00018 #include <deque>
00019 #include <exception>
00020 #include <functional>
00021 #include <memory>
00022 #include <mutex>
00023 #include <print>
00024 #include <stdexcept>
00025 #include <string>
00026 #include <thread>
00027 #include <tuple>
00028 #include <type_traits>
00029 #include <utility>
00030 #include <vector>
00031
00032 namespace untangle {
00033
00045 template <typename actionT>
00046 class executor {
00047 public:
00056     explicit executor(std::size_t worker_count) {
00057         if (worker_count == 0) {
00058             throw std::invalid_argument("executor: worker_count must not be zero");
00059         }
00060
00061         workers_.reserve(worker_count);
00062
00063         for (std::size_t index = 0; index < worker_count; ++index) {
00064             auto next = worker_execution::create_instance("pool_worker_" + std::to_string(index));
00065
00066             // Tells the pool the moment this worker has room for more.
00067             next->on_finished = [this, index] { take_next_task(index); };
00068
00069             // Carries what a task throws to the caller. Assigned here because a worker thread reads it.
00070             next->on_error = [this, name = next->name](std::exception_ptr thrown) {
00071                 report_task_error(name, thrown);
00072             };
00073
00074             // Adds the worker to the poll the destructor waits on.
00075             poll_.add(*next);
00076
00077             workers_.push_back(std::move(next));
00078         }
00079     }
00080
00095     ~executor() {
00096         // Whether the pool was still running when it was destroyed. A stopped one cannot empty its
00097         // queue - only a worker takes from it, through on_finished, and a stopped worker raises no more
```

```

00098 // - so its queue is not waited for.
00099 const bool was_running = running_.exchange(false);
00100
00101 {
00102     std::unique_lock<std::mutex> lock(mutex_);
00103
00104     // Waits for the queue to empty and every worker to go idle.
00105     auto interval = std::chrono::milliseconds(report_first_ms);
00106     auto waited = std::chrono::milliseconds(0);
00107
00108     while (!finished_cv_.wait_for(lock, interval, [this, was_running] {
00109         return nothing_running() && (pending_.empty() || !was_running);
00110     })) {
00111         waited += interval;
00112
00113         // Under the lock, so the counts agree with the predicate that just failed.
00114         std::println(
00115             stderr, "executor: still waiting to finish after {}s - {} queued, {} in a task{}",
00116             waited.count() / 1000, pending_.size(), count_workers(&worker_execution::is_busy),
00117             name_workers(&worker_execution::is_busy));
00118
00119         interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00120     }
00121
00122     if (!pending_.empty()) {
00123         std::println(stderr,
00124             "executor: {} queued task(s) dropped, the pool was stopped before they ran",
00125             pending_.size());
00126     }
00127 }
00128
00129 stop();
00130
00131 // Waits for every worker to leave its thread, which is what makes destroying them safe. A
00132 // different question from the wait above: a worker can be idle and still be in its thread.
00133 auto interval = std::chrono::milliseconds(report_first_ms);
00134 auto waited = std::chrono::milliseconds(0);
00135
00136 while (poll_.is_running()) { // time of check
00137     const auto tick = std::chrono::milliseconds(poll_interval_ms);
00138     std::this_thread::sleep_for(tick);
00139     waited += tick;
00140
00141     if (waited >= interval) {
00142         std::println(stderr, "executor: still waiting to stop after {}s - {} not left its thread{}",
00143             waited.count() / 1000, count_workers(&worker_execution::is_running),
00144             name_workers(&worker_execution::is_running));
00145
00146         interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00147     }
00148 }
00149
00150 // time of use: no worker thread is left to reach workers_.
00151 workers_.clear();
00152 }
00153
00154 template <typename... Args>
00155 bool add_action(actionT task, Args&&... args) {
00156     // The call the pool will make, which is not quite the one written at the call site: what
00157     // reaches the task is the copies bound below, so that is the call that has to compile.
00158     static_assert(std::is_invocable_v<actionT, std::decay_t<Args>&...>,
00159         "executor: the task cannot be called with the arguments given to add_action()");
00160
00161     if (!running_) {
00162         return false;
00163     }
00164
00165     // Bound before the lock: it copies the caller's arguments, and nothing here needs the queue.
00166     return queue_task(bind_action(std::move(task), std::forward<Args>(args)...));
00167 }
00168
00169 template <typename... Args>
00170 bool add_task(actionT task, Args&&... args) {
00171     if (!running_) {
00172         return false;
00173     }
00174
00175     // untangle::bind_task() takes the callback off the end of the pack and checks it; what comes
00176     // back is one callable that runs the work and then notifies, which is what the queue holds.
00177     auto action = untangle::bind_task(std::move(task), std::forward<Args>(args)...);
00178
00179     // An empty one is what bind_task() builds from an empty action or an empty callback. Queued, it
00180     // would reach a worker and throw std::bad_function_call there, reporting a failure to whoever
00181     // set on_task_error rather than to the caller who could still act on it. The same refusal
00182     // untangle::actuator::add_task() makes, made before the queue rather than after.
00183     if (!action) {
00184         return false;
00185     }
00186 }

```

```

00240     }
00241
00242     return queue_task(std::move(action));
00243 }
00244
00248 std::size_t pending() const {
00249     std::lock_guard<std::mutex> lock(mutex_);
00250     return pending_.size();
00251 }
00252
00253 void start() {
00261     // The workers first, so anything accepted after this is handed to one that is running.
00262     for (auto& one : workers_) {
00263         one->start();
00264     }
00265
00266     running_ = true;
00267 }
00268
00269 void stop() {
00280     // First, so that nothing is accepted for workers that are about to stop. A task already on a
00281     // worker still runs; one arriving from here is refused.
00282     running_ = false;
00283
00284     // No lock: a worker waking from stop() runs on_finished, which takes mutex_.
00285     for (auto& one : workers_) {
00286         one->stop();
00287     }
00288 }
00289
00307 std::function<void(std::exception_ptr)> on_task_error;
00308
00309 private:
00316 void report_task_error(const std::string& worker, std::exception_ptr thrown) {
00317     if (on_task_error) {
00318         on_task_error(thrown);
00319         return;
00320     }
00321
00322     try {
00323         std::rethrow_exception(thrown);
00324     } catch (const std::exception& e) {
00325         std::println(stderr, "warning: executor task on '{}' threw: {}", worker, e.what());
00326     } catch (...) {
00327         std::println(stderr, "warning: executor task on '{}' threw an unknown type", worker);
00328     }
00329 }
00330
00341 template <typename... Args>
00342 static untangle::task_t bind_action(actionT task, Args&&... args) {
00343     if constexpr (sizeof...(Args) == 0) {
00344         return untangle::task_t(std::move(task));
00345     } else {
00346         // Captured by value and called as lvalues, which is all a call deferred to a worker can
00347         // promise: whatever was passed to add_action() may be gone by the time this runs.
00348         //
00349         // One tuple rather than an init-capture pack: a pack expanded into a lambda's init-capture
00350         // inside a dependent context is an ICE in GCC 14 (tsubst_pack_expansion), and std::apply says
00351         // the same thing in an ordinary expression. untangle::bind_task() was moved off the same
00352         // shape for the same reason.
00353         return untangle::task_t([task = std::move(task), args = std::tuple<std::decay_t<Args>...>(
00354             std::forward<Args>(args)...)]() mutable {
00355             std::apply(task, args);
00356         });
00357     }
00358 }
00359
00360 using worker_execution = untangle::async::execution<untangle::task_t>;
00361
00370 bool queue_task(untangle::task_t task) {
00371     std::lock_guard<std::mutex> lock(mutex_);
00372
00373     // The queue comes first: an entry does not overtake one already waiting in it.
00374     if (pending_.empty()) {
00375         // Always from the start, so a free pool reuses its warmest worker rather than waking a cold
00376         // one. Under load the workers are busy and the queue below is what spreads the work.
00377         for (std::size_t index = 0; index < workers_.size(); ++index) {
00378             if (!workers_[index]->is_busy()) {
00379                 // A refused entry is already destroyed, and stop() stops every worker together, so there
00380                 // is nothing to queue and no other worker to try.
00381                 return give_to_worker(index, std::move(task));
00382             }
00383         }
00384     }
00385
00386     pending_.push_back(std::move(task));
00387     return true;

```

```

00388 }
00389
00401 [[nodiscard]] bool give_to_worker(std::size_t index, untangle::task_t task) {
00402     // The door takes the worker's action_mutex under mutex_. The two never cycle, because a worker
00403     // calls on_finished with its action_mutex released.
00404     return workers_[index]->add_action(std::move(task));
00405 }
00406
00410 void take_next_task(std::size_t index) {
00411     std::lock_guard<std::mutex> lock(mutex_);
00412
00413     if (!pending_.empty()) {
00414         untangle::task_t next = std::move(pending_.front());
00415         pending_.pop_front();
00416
00417         if (!give_to_worker(index, std::move(next))) {
00418             // The task is already destroyed, so it is reported rather than put back.
00419             std::println(stderr, "executor: {} refused a queued task, which is lost",
00420                 workers_[index]->name);
00421         }
00422
00423         return;
00424     }
00425
00426     if (nothing_running()) {
00427         finished_cv_.notify_all();
00428     }
00429 }
00430
00436 bool nothing_running() const {
00437     for (const auto& one : workers_) {
00438         if (one->is_busy()) {
00439             return false;
00440         }
00441     }
00442
00443     return true;
00444 }
00445
00451 std::size_t count_workers(bool (worker_execution::*state)() const) const {
00452     std::size_t count = 0;
00453
00454     for (const auto& one : workers_) {
00455         if ((one.get()->*state)()) {
00456             ++count;
00457         }
00458     }
00459
00460     return count;
00461 }
00462
00469 std::string name_workers(bool (worker_execution::*state)() const) const {
00470     std::string names;
00471
00472     for (const auto& one : workers_) {
00473         if ((one.get()->*state)()) {
00474             names += names.empty() ? " : " : ", ";
00475             names += one->name;
00476         }
00477     }
00478
00479     return names;
00480 }
00481
00483 static constexpr int poll_interval_ms = 50;
00484
00486 static constexpr int report_first_ms = 1000;
00487 static constexpr int report_max_ms = 30000;
00488
00489 mutable std::mutex mutex_;
00490 std::condition_variable finished_cv_;
00491
00497 async::execution_poll poll_;
00498
00500 std::deque<untangle::task_t> pending_;
00501 std::vector<std::shared_ptr<worker_execution> > workers_;
00503 std::atomic_bool running_ = {false};
00504 };
00505
00506 } // namespace untangle

```

Index

- ~executor
 - untangle::executor< actionT >, [16](#)
- add_action
 - untangle::executor< actionT >, [17](#)
- add_task
 - untangle::executor< actionT >, [17](#)
- executor, [1](#)
 - untangle::executor< actionT >, [16](#)
- on_task_error
 - untangle::executor< actionT >, [19](#)
- start
 - untangle::executor< actionT >, [18](#)
- stop
 - untangle::executor< actionT >, [19](#)
- untangle, [13](#)
- untangle::executor< actionT >, [15](#)
 - ~executor, [16](#)
 - add_action, [17](#)
 - add_task, [17](#)
 - executor, [16](#)
 - on_task_error, [19](#)
 - start, [18](#)
 - stop, [19](#)